INFORMATION SYSTEMS INTEGRATION

TP01 – Final Report

Formula 1 Session Data: ETL and Integration

COURSE UNIT	Information Systems Integration
ACADEMIC YEAR	2026/2027
AUTHOR	Robert Andone
STUDENT NUMBER	51242
TOOLS	Pentaho Data Integration (Spoon) 10.2

1. Context

This project applies ETL (Extract, Transform and Load) to a collection of Formula 1 session data for the 2026 season. The files describe different parts of a race weekend, including circuits, meetings, sessions, drivers, lap timing, session results, tyre stints and pit stops.

Pentaho Data Integration (PDI/Kettle) performs the cleaning, joining and aggregation steps. The workflow uses CSV datasets for circuits, meetings, sessions, drivers, lap timing, session results, tyre stints and pit stops. Each transformation writes inspectable CSV outputs, including rejected-data files for records that do not meet the required checks.

2. Problem

The datasets describe different parts of a race weekend and have different levels of detail. A lap belongs to a driver in a particular session, while a session belongs to a meeting; pit-stop and stint records have their own keys and optional measurements. Combining the files therefore depends on consistent field names, appropriate key types and lookups that use enough fields to identify a record.

The aim was to build a repeatable workflow that cleans the inputs, removes duplicate rows, joins related entities and produces a summary of timed laps by session and driver. Missing optional measurements also need careful handling: a blank speed, tyre-age or duration value should not cause the whole transformation to fail or be treated as a real measurement.

3. Strategy used

The workflow uses CSV as its input and output format. The source datasets are stored in data/input and cover circuits, meetings, sessions, drivers, lap timing, session results, tyre stints and pit stops. Pentaho transformations clean the records, standardise key fields, remove duplicates, combine related datasets and produce summary outputs in data/output.

Source file(s)	Format	Use in the workflow
`circuits.csv`	CSV	Clean circuit reference records.
`session_results_2026.csv`	CSV	Clean session result rows before joining driver details.
`laps_2026.csv`	CSV	Clean lap and sector timing data; later calculate session/driver metrics.
`stints_2026.csv`	CSV	Clean tyre stint records and add driver/team details.
`pit_stops_2026.csv`	CSV	Clean pit-stop records and add session information.
`sessions_2026.csv`, `drivers_2026.csv`, `meetings_2026.csv`	CSV	Clean session, driver and meeting records.

Data quality checks focus on the fields required for integration, consistent session and driver keys, duplicate records and usable lap-duration values. Records rejected by these checks are written to separate output files so that the decisions remain visible and can be reviewed.



Figure 1. Main data flow from local inputs to the generated CSV outputs.

4. Transformations

The cleaning transformations follow a similar pattern, although the exact fields and filters vary by dataset. String Operations are used to tidy text, Select Values is used where the schema needs adjustment, and Filter Rows, Sort Rows and Unique Rows handle validation and duplicate removal. Where a filter rejects a row, the transformation writes it to a separate rejected-data file.

Transformation	Purpose and main operations	Output
`01_clean_circuits`	Trim/normalise fields; select columns; filter, sort and remove duplicates.	`circuits_clean.csv`; `circuits_rejected.csv`
`02_clean_sessions`	Read session CSV; filter missing session keys; sort and deduplicate by session.	`sessions_clean.csv`; `sessions_rejected.csv`
`03_clean_drivers`	Read driver CSV; normalise text and remove duplicate records.	`drivers_clean.csv`
`04_clean_session_results`	Require the keys needed for joining; sort and deduplicate session/driver results.	`session_results_clean.csv`; `session_results_rejected.csv`
`05_clean_laps`	Filter records with missing session keys; sort and deduplicate by session, driver and lap.	`laps_clean.csv`; `laps_rejected.csv`
`06_clean_stints`	Filter missing session keys; sort and deduplicate stint records.	`stints_clean.csv`; `stints_rejected.csv`
`07_clean_pit_stops`	Filter missing session keys; sort and remove duplicate session/driver/lap records.	`pit_stops_clean.csv`; `pit_stops_rejected.csv`
`08_clean_meetings`	Read meeting CSV; require meeting keys; sort and deduplicate by meeting.	`meetings_clean.csv`; `meetings_rejected.csv`

4.1 Joins and lookups

The integration transformations use Stream Lookup to append related information. Driver lookups use both session_key and driver_number, rather than driver_number alone, because driver numbers can appear in more than one session. The pit-stop input key is exposed as pit_session_key in that transformation and is matched to session_key in the sessions data.

Transformation	Lookup	Key fields	Output
09 - Results + drivers	Driver details	`session_key + driver_number`	`session_results_with_drivers.csv`
10 - Laps + drivers	Driver/team details	`session_key + driver_number`	`laps_with_drivers.csv`
11 - Stints + drivers	Driver/team details	`session_key + driver_number`	`stints_with_drivers.csv`
12 - Pit stops + sessions	Session name, type and dates	`pit_session_key → session_key`	`pit_stops_with_sessions.csv`

Some optional speed and tyre-age fields are kept as text in the joined outputs. This preserves blanks from the source files and avoids type errors when a field contains whitespace instead of a number.

4.2 Session summary

Transformation 13 reads laps_with_drivers.csv and keeps rows with a usable lap_duration before sorting and grouping. Group By uses session_key, driver_number, full_name and team_name as the grouping fields. It calculates timed_lap_count, average_lap_duration and fastest_lap_duration, where fastest_lap_duration is the minimum available lap duration in the group. Rows without a usable duration are routed to untimed_laps_rejected.csv.

The count represents timed lap records, not every row in laps_clean.csv. The current summary does not attempt to remove pit-out laps or other unusually slow laps, so its average should be read as a basic descriptive measure rather than a clean race-pace estimate.

4.3 Results and checks

The bundled output files provide a concrete result of the workflow. The counts below were measured from the CSV files included in the supplied project archive. They should be checked again after running the packaged master Job in Spoon, because that run regenerates the outputs.

Output file	Data rows	Notes
circuits_clean.csv	77	Circuit reference records.
sessions_clean.csv	131	Unique session records.
meetings_clean.csv	27	Unique meeting records.
drivers_clean.csv	10,059	Driver/session rows.
session_results_clean.csv	1,989	Cleaned session results.
laps_clean.csv	56,703	Cleaned lap rows.
stints_clean.csv	8,162	Cleaned tyre stint rows.
pit_stops_clean.csv	7,684	After duplicate removal; the supplied raw CSV has 8,622 records.
laps_with_drivers.csv	56,703	Lap rows with driver/team information.
stints_with_drivers.csv	8,162	Stint rows with driver/team information.
pit_stops_with_sessions.csv	7,684	Pit-stop rows with session information.
session_results_with_drivers.csv	1,989	Result rows with driver information.
session_summary.txt	1,946	One row per session/driver/team group with timed laps.

In the bundled session summary, one example row is session 11227, driver 1 (Lando Norris, McLaren): 6 timed laps, an average lap duration of 253.112 seconds, and a minimum lap duration of 84.391 seconds. The average is high because the current calculation includes every lap record with a usable duration; it is not intended as an official performance statistic.

The bundled lap data contain 2,134 rows with blank lap_duration values. The summary includes 54,569 timed laps, and transformation 13 is configured to route untimed records to untimed_laps_rejected.csv. Confirm that this file is generated by the final Spoon run before submission.

5. Jobs

The master Job is dataintegration/00_run_etl.kjb. It coordinates the source-cleaning transformations, the four integration transformations and the session summary. The hops are configured to continue on success, so a failed stage should prevent later stages from running.

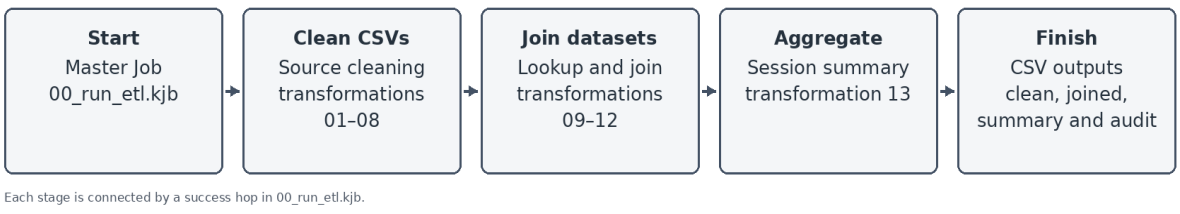


Figure 2. Simplified execution order of the master Job.

The configured order is: 01_clean_circuits; 02_clean_sessions; 03_clean_drivers; 04_clean_session_results; 05_clean_laps; 06_clean_stints; 07_clean_pit_stops; 08_clean_meetings; 09_combine_sessions_drivers; 10_combine_laps_drivers; 11_combine_stints_drivers; 12_combine_pit_stops_sessions; and 13_session_summary.

To run the project, extract the tp01-51242 directory without changing its internal structure, open dataintegration/00_run_etl.kjb in Spoon, and use the local run configuration. The transformation paths are relative to their own files, so the folder can be moved as a unit. Before submission, run the packaged Job in Spoon and confirm that every entry succeeds.

6. Video demonstration

The video should show the extracted project folder, the master Job canvas, a complete successful run, one cleaning transformation, one lookup and the final session_summary.txt file. It should briefly explain the purpose of the ETL stages and where the output files are saved.

Video URL: https://youtu.be/wczcOYMRy9k

7. Conclusion and future work

The project brings together several Formula 1 datasets that would otherwise have to be inspected separately. The workflow cleans the CSV inputs, removes duplicate records, adds driver or session details through lookups and calculates lap-duration summaries. Rejected-data outputs make data-quality decisions visible rather than silently discarding records.

There are clear ways to extend it. The current workflow uses local snapshots rather than requesting fresh data from the OpenF1 API at run time. A relational database could be added as a destination, and the summary could be improved by separating pit-out laps and other non-representative timing records. Further checks could report lookup match rates and compare output schemas automatically after each run.

8. References

[1] OpenF1, API documentation: openf1.org/docs.

[2] Pentaho, Pentaho Data Integration 10.2 documentation.

[3] Pentaho, Pentaho Data Integration 10.2 documentation.